

Listes Imbriquées

- On peut stocker **tout** dans une liste, aussi des listes!
- Cela est pratique, par exemple pour stocker une matrice / tableau

1.0	2.0	3.0
3.0	4.0	5.0

```
matrix = [ [ 1.0, 2.0 , 3.0 ], [ 3.0, 4.0, 5.0] ]
```

- Comment parcourir la matrice?

Listes Imbriquées

	j=0	j=1	j=2
i=0	1.0	2.0	3.0
i=1	3.0	4.0	5.0

```
matrix = [ [ 1.0, 2.0 ,3.0 ], [ 3.0, 4.0, 5.0] ]
```

```
for row in matrix:  
    for value in row:  
        print(value)
```

row=[1.0,2.0,3.0]	value=1.0	1.0
	value=2.0	2.0
	value=3.0	3.0
row=[3.0,4.0,5.0]	value=3.0	3.0
	value=4.0	4.0
	value=5.0	5.0

```
for i in range(len(matrix)):  
    for j in range(len(matrix[i])):  
        print(matrix[i][j])
```

i=0	j=0	1.0
	j=1	2.0
	j=2	3.0
i=1	j=0	3.0
	j=1	4.0
	j=2	5.0

Création de Matrices

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = [ [ 0.0, 0.0 , 0.0 ], [ 0.0, 0.0, 0.0] ]
```

```
matrix = []
```

```
for i in range(2):
```

```
    row = []
```

```
    for j in range(3):
```

```
        row.append(0.0)
```

```
    matrix.append ( row )
```

matrix=[]

i=0

row = []

j=0

row = [0.0]

j=1

row = [0.0,0.0]

j=2

row = [0.0,0.0,0.0]

Création de Matrices

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = [ [ 0.0, 0.0 , 0.0 ] , [ 0.0, 0.0, 0.0] ]
```

```
matrix = []
```

```
for i in range(2):
```

```
    row = []
```

```
    for j in range(3):
```

```
        row.append(0.0)
```

```
    matrix.append ( row )
```

matrix=[[0.0,0.0,0.0]]

i=1

row = []

j=0

row = [0.0]

j=1

row = [0.0,0.0]

j=2

row = [0.0,0.0,0.0]

Création de Matrices

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = [ [ 0.0, 0.0 , 0.0 ] , [ 0.0, 0.0, 0.0] ]
```

```
matrix = []
```

```
for i in range(2):
```

```
    row = []
```

```
    for j in range(3):
```

```
        row.append(0.0)
```

```
    matrix.append ( row )
```

```
matrix=[[0.0,0.0,0.0],[0.0,0.0,0.0]]
```

Création de Matrices

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = [ [ 0.0, 0.0 , 0.0 ], [ 0.0, 0.0, 0.0] ]
```

```
matrix = []
```

```
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(0.0)  
    matrix.append ( row )
```

```
matrix = [ [ 0.0 for j in range(3) ] for i in range(2) ]
```

Création de Matrices

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = [ [ 0.0, 0.0 , 0.0 ], [ 0.0, 0.0, 0.0] ]
```

```
matrix = []
```

```
for i in range(2):
```

```
    row = []
```

```
    for j in range(3):
```

```
        row.append(0.0)
```

```
    matrix.append ( row )
```

```
matrix = [ [ 0.0 for j in range(3) ] for i in range(2) ]
```

Enumerate

- Certaines fonctions en Python créent des listes imbriquées
- Bon exemple: `enumerate`

```
l = [ "Bruxelles", "Mons", "Anvers" ]  
for t in enumerate(l):  
    print ( t )
```

(0,"Bruxelles")

(1,"Mons")

(2,"Anvers")

Enumerate

Il y a une notation plus courte qui le rend plus facile de parcourir des listes imbriquées de longueur fixe

```
l = [ "Bruxelles", "Mons", "Anvers" ]  
for index, city in enumerate(l):  
    print ( "City", index, ":", city )
```

Tuple dans la liste	variables		imprimé
(0,"Bruxelles")	index=0	city="Bruxelles"	City 0 : Bruxelles
(1,"Mons")	index=1	city="Mons"	City 1 : Mons
(2,"Anvers")	index=2	city="Anvers"	City 2 : Anvers

Complications avec Objets

- Pour créer une matrice on a vu:

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = []
```

```
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(0.0)  
    matrix.append ( row )
```

Complications avec Objets

- Pourquoi pas :

0.0	0.0	0.0
0.0	0.0	0.0

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```

```
matrix = []  
  
for i in range(2):  
    row = []  
    for j in range(3):  
        row.append(0.0)  
    matrix.append ( row )
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```

- À première vue cela paraît correct:

```
print ( matrix )
```

```
[[0.0,0.0,0.0],[0.0,0.0,0.0]]
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```

- Mais...

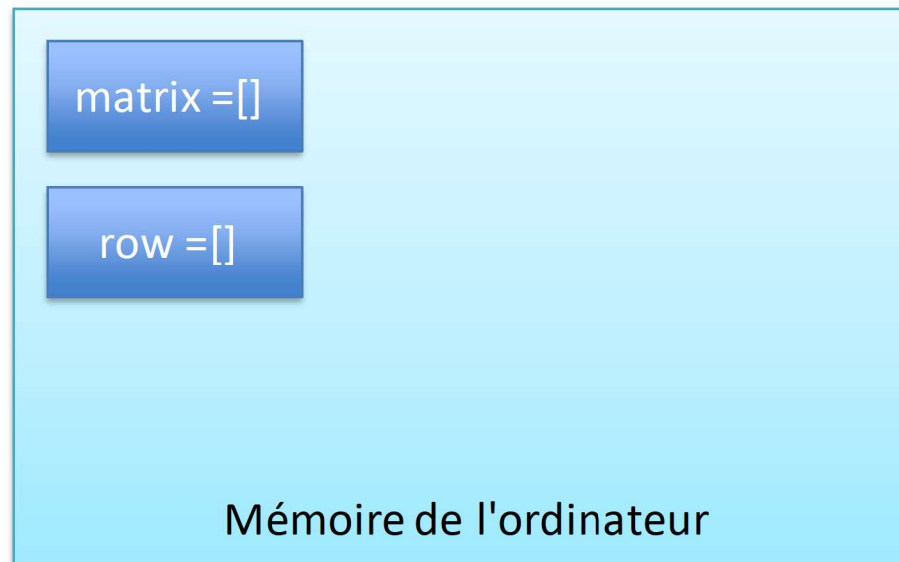
```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[1.0,0.0,0.0]]
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```



- Mais...

```
matrix[0][0] = 1.0  
print ( matrix )
```

A red rectangular box containing the text '[[1.0,0.0,0.0],[1.0,0.0,0.0]]'. The first '1.0' in each row is highlighted in yellow.

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```

matrix=[]

row=[0.0,0.0,0.0]

Mémoire de l'ordinateur

- Mais...

```
matrix[0][0] = 1.0  
print ( matrix )
```

[[1.0,0.0,0.0],[1.0,0.0,0.0]]

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```



- Mais...

```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0, 0.0, 0.0], [1.0, 0.0, 0.0]]
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```



- Mais...

```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0, 0.0, 0.0], [1.0, 0.0, 0.0]]
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```



- Mais...

```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0, 0.0, 0.0], [1.0, 0.0, 0.0]]
```

Complications avec Objets

- Pourquoi pas :

```
matrix = []  
row = []  
for j in range(3):  
    row.append(0.0)  
  
for i in range(2):  
    matrix.append ( row )
```



- Mais...

```
row[1] = 2.0  
print ( matrix )
```

```
[[1.0, 2.0, 0.0], [1.0, 2.0, 0.0]]
```

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
    for j in range(3):  
        row.append(0.0)
```

matrix=[]

row=[]

Mémoire de l'ordinateur

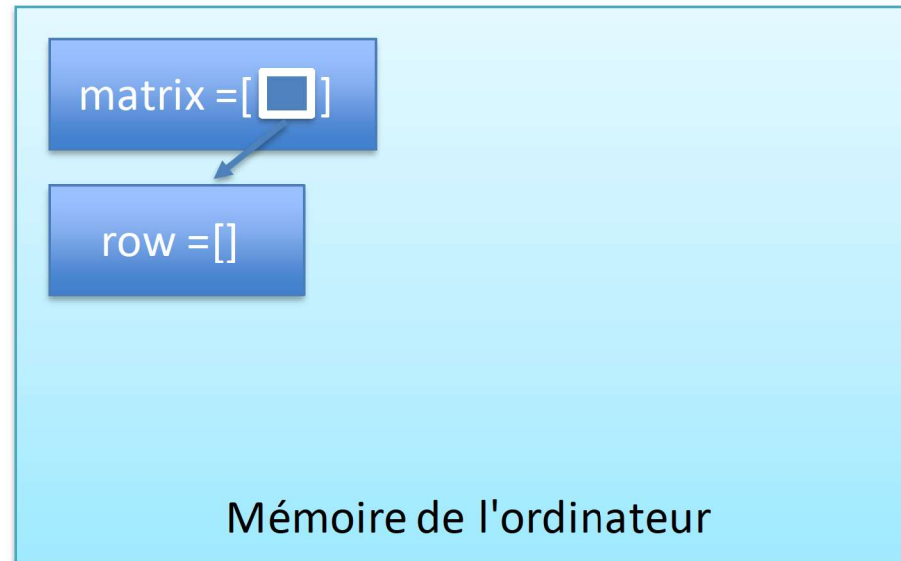
```
matrix[0][0] = 1.0  
print ( matrix )
```

[[1.0,0.0,0.0],[0.0,0.0,0.0]]

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
for j in range(3):  
    row.append(0.0)
```



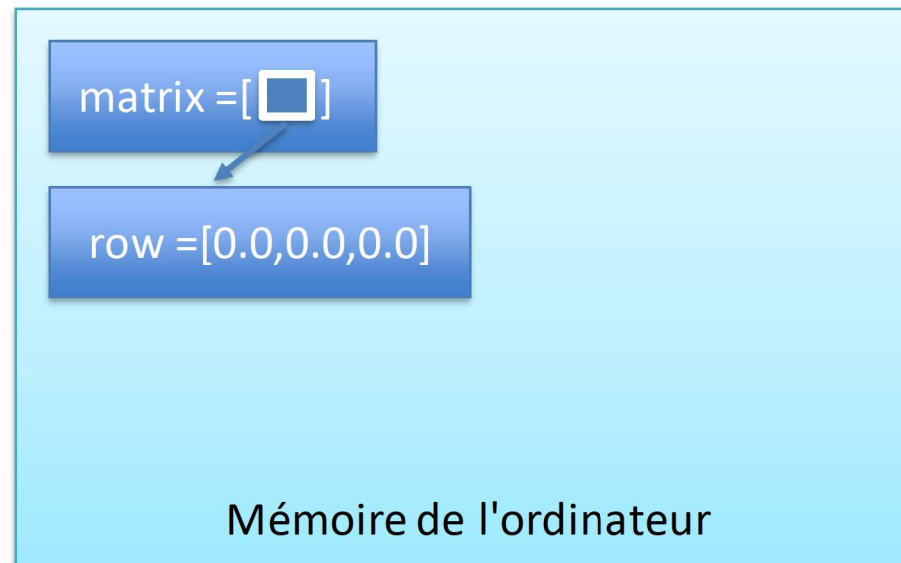
```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[0.0,0.0,0.0]]
```

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
for j in range(3):  
    row.append(0.0)
```



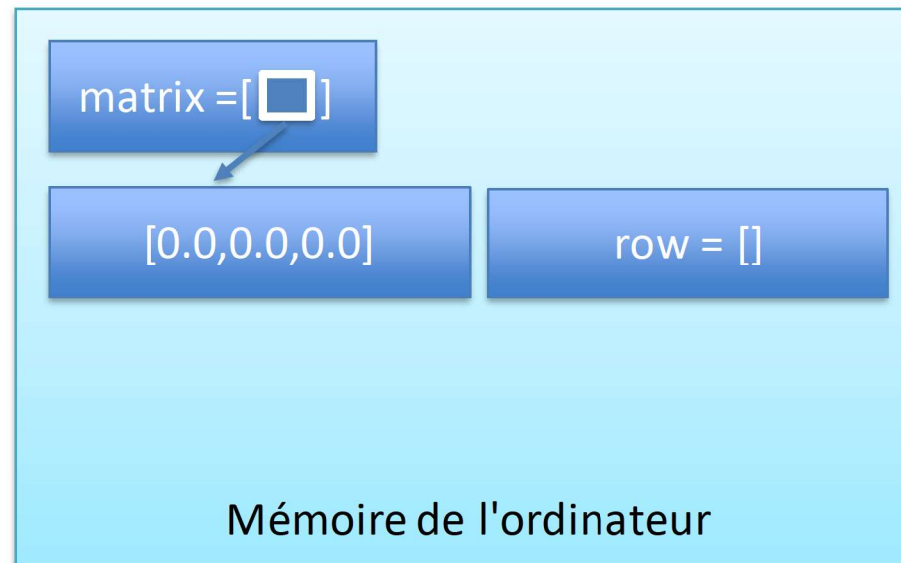
```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[0.0,0.0,0.0]]
```

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
for j in range(3):  
    row.append(0.0)
```



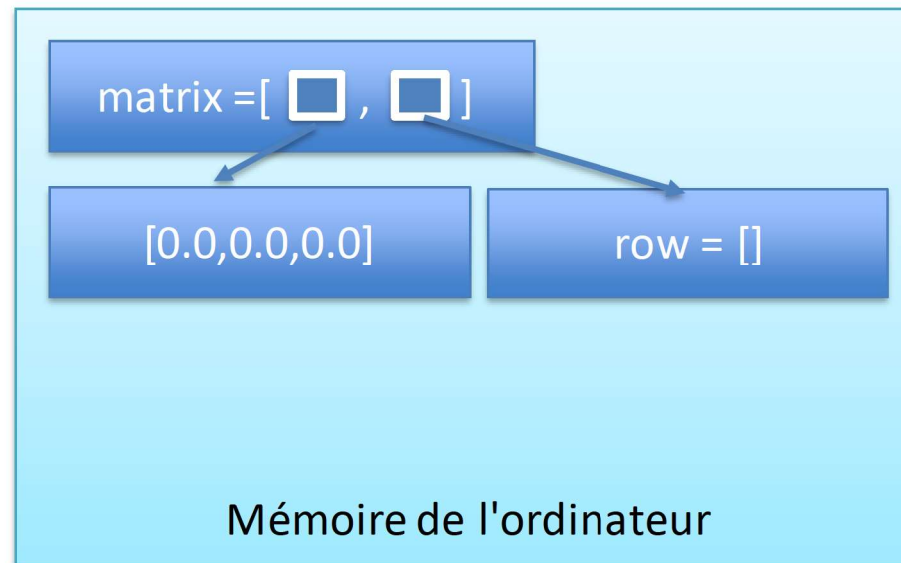
```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[0.0,0.0,0.0]]
```

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
    for j in range(3):  
        row.append(0.0)
```



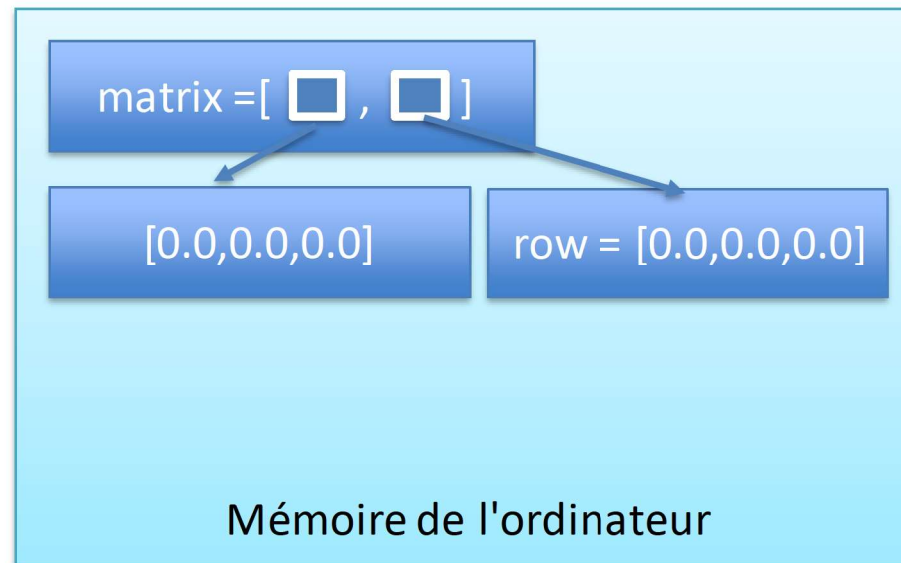
```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[0.0,0.0,0.0]]
```

Complications avec Objets

- Correct :

```
matrix = []  
for i in range(2):  
    row = []  
    matrix.append ( row )  
for j in range(3):  
    row.append(0.0)
```



```
matrix[0][0] = 1.0  
print ( matrix )
```

```
[[1.0,0.0,0.0],[0.0,0.0,0.0]]
```